

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-241
Караульний Максим Миколайович

Перевірів: ас. Баранов О.М.

Одеса – 2026

Звіт з лабораторної роботи №1

Завдання 1. Фактичне питання та задача на міркування

Фактичне питання

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ.
eval_count	346	5
Час, с	23.88	80.51
Токенів/с	15.66	33.77

Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	1 (з 1)	0 (з 1)
eval_count	1608	256
Час, с	129.93	14.03

Відповіді на питання:

- Куди пішли токени в міркувальній моделі?** Токени були витрачені на генерацію внутрішнього "ланцюжка думок" (chain of thought), де модель перевіряла факти перед тим, як видати остаточну відповідь.
- Де міркувальні моделі дають перевагу?** Вони кращі для математичних задач, логічних головоломок та програмування, але їх використання невиправдане для простих фактичних питань.
- Чим відрізняється навчання?** Міркувальні моделі додатково тренуються на датасетах із покроковими розв'язаннями (наприклад, за допомогою RLHF) для формування прихованих думок.
- Більше токенів = розумніша?** Ні, більша кількість токенів означає лише більш детальний процес мислення, але не гарантує правильність чи вищий інтелект бази.

5. **Різниця у швидкості (токенів/с)?** Швидкість залежить від архітектури моделі (розмір словника, кількість шарів) та її розміру в параметрах (3B генерує швидше за 4B).

Власні думки: Міркувальні моделі ідеальні для складних задач, але на простих питаннях вони лише марнують ресурси процесора та час очікування.

Завдання 2. Вплив system-повідомлення

Вплив system-повідомлення

Модель	System	Довжина відповіді	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	2 слова	укр	1	Ні (відповіла 2 словами)
llama3.2:3b	українською, 2 речення	34 слова	укр	2	Так
qwen3:4b	лаконічний асистент	1 слово	укр	1	Так (але з помилкою у факті)
qwen3:4b	українською, 2 речення	30 слів	укр	2	Так

Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	51	9	60
llama3.2:3b	українською, 2 речення	60	87	147
qwen3:4b	лаконічний асистент	75	1782	1857
qwen3:4b	українською, 2 речення	51	2790	2841

Відповіді на питання:

1. **Яка модель точніше дотрималась обмеження?** Обидві моделі непогано впоралися, проте llama3.2:3b рахує речення краще, оскільки є спеціалізованою інструктивною моделлю.

- 2. Чи були випадки відповіді не українською? Малі моделі іноді можуть збиватися на англійську або російську через недостатню кількість українських текстів у навчальній вибірці (особливо при складних запитах).
- 3. Як змінився `prompt_tokens`? Він збільшився рівно на ту кількість токенів, на яку подовжилося `system`-повідомлення.
- 4. Чому в `qwen3:4b` більше `completion_tokens`? Тому що це міркувальна модель, яка генерує приховані токени роздумів (`thinking`) перед самою відповіддю.

Питання до чат-бота:

- 1. Чому моделі по-різному дотримуються інструкцій? Бо вони мають різні архітектури та обсяги навчальних даних щодо слідування інструкціям (`instruction tuning`).
- 2. Пріоритет `system`-повідомлення: Так, `system`-повідомлення зазвичай має найвищий пріоритет у структурі уваги (`attention`) моделі.
- 3. Вихідні токени в міркувальній: Вона витрачає їх на "чорновик" (ланцюжок думок), який користувач не бачить.
- 4. Суперечливі інструкції: Залежить від конкретної моделі, але частіше перемагає `system`, оскільки вона сприймається як базове правило.
- 5. Малі моделі та мова: Малі моделі мають менший обсяг знань (параметрів), тому їхня здатність "думати" однією мовою, а відповідати іншою — значно слабша.

Власні думки: `System`-повідомлення є потужним інструментом, але малі моделі не завжди ідеально його дотримуються. Для суворих форматів однієї інструкції часто замало.

Завдання 3. Порівняння локальної та хмарної моделі

Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.5-flash-lite
Час відповіді, с	13.29	1.27
<code>completion_tokens</code>	82	221
Довжина відповіді, символів	~260	~500
Мова відповіді	суржик/укр	укр
Фактичних помилок у відповіді	2	0

Відповіді на питання:

- 1. **Скільки рядків змінено?** Жодного в основній логіці `compare.py`, змінився лише словник конфігурацій `PROVIDERS`.
- 2. **Яка повніша?** Хмарна модель дає більш структуровану, глибоку і точну відповідь завдяки більшому розміру (8B параметрів проти 3B).
- 3. **Проблеми з українською в локальній?** У локальної моделі іноді проскакували русизми (наприклад, "списки можуть бути змінені").
- 4. **Чому хмарна швидша?** Вона працює на потужних промислових графічних процесорах (GPU) або спеціалізованих чіпах (як LPU у Groq), які генерують токени в сотні разів швидше за домашній комп'ютер.

Питання до чат-бота:

- 1. **Чому OpenAI стандарт?** Бо вони першими захопили ринок з ChatGPT, і всім іншим було вигідно зробити сумісний API для легкого переходу клієнтів.
- 2. **Розмір моделі:** Розмір (кількість параметрів) є ключовим фактором, що визначає загальну ерудицію та логіку моделі.
- 3. **Швидкість хмарної:** Промислові сервери з оптимізованим обладнанням компенсують затримку мережі надвисокою швидкістю інференсу.
- 4. **Ризики приватності:** Дані йдуть на чужі сервери. У комерції укладають B2B-угоди (наприклад, Azure OpenAI), де провайдер гарантує, що не тренуватиме моделі на даних клієнта.
- 5. **Rate limit:** Це обмеження кількості запитів на хвилину. Розробники обходять його кешуванням, паузами (sleep) або балансуванням між кількома ключами.

Власні думки: Локальну варто обирати для роботи з конфіденційними даними (наприклад, медичними), а хмарну – для всього іншого, де потрібна швидкість і високий інтелект.

Завдання 4. Вимірювання швидкості

Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.17	2.72	16.6
локальна	llama3.2:3b	середній	2.17	4.66	62.3
локальна	llama3.2:3b	довгий	2.22	9.60	88.5
хмарна	gemini-3.5-flash-lite	короткий	0.60	0.64	37.7
хмарна	gemini-3.5-flash-lite	середній	0.70	1.20	380.5

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
хмарна	gemini-3.5-flash-lite	довгий	0.71	3.13	877.3

Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	~35.07	2.17
Всього, с	~80.51	2.72

Відповіді на питання:

- 1. **У кого кращий TTFT?** У локальної моделі (на теплому старті), бо немає затримки на передачу даних через інтернет (пінг).
- 2. **Чи збігаються найшвидші?** Ні. Локальна швидше починає друкувати (кращий TTFT), але хмарна генерує текст (симв/с) набагато швидше.
- 3. **Співвідношення при короткому/довгому промпті:** На короткому виграє локальна (через швидкий старт), на довгому беззаперечно перемагає хмарна (за рахунок високої швидкості генерації).
- 4. **Розбіжність повторів:** Три запуски потрібні для згладжування випадкових піків навантаження мережі або процесора. Одного запуску недостатньо через високу дисперсію мережевих затримок.
- 5. **Чому холодний старт повільніший?** Бо модель спочатку вантажиться з жорсткого диска (HDD/SSD) в оперативну/відео пам'ять (RAM/VRAM). Теплий старт використовує вже завантажену модель.

Питання до чат-бота:

- 1. **TTFT фізично:** Це час на обробку всього вхідного промпту (Prompt Processing). Вимірюють окремо, бо це впливає на перше враження користувача (наскільки швидко "оживає" бот).
- 2. **Гірший TTFT у хмари:** Через фізичну відстань пакетів по мережі інтернет.
- 3. **Холодний старт:** Це процес читання гігабайтів ваг моделі з диска в RAM/VRAM.
- 4. **Медіана:** Дозволяє відкинути аномалії (наприклад, якщо один запит завис через антивірус або стрибок пінгу).
- 5. **Обробка швидше генерації:** Бо вхідний промпт обробляється паралельно (всі токени одночасно через матричні множення), а генерація — послідовно (по одному токену).

Власні думки: Для інтерактивного чат-бота найважливіший TTFT (користувач не любить чекати відповіді), а для фонової обробки документів (наприклад, аналіз тисячі відгуків) — загальна швидкість генерації (симв/с).

Завдання 5. Оцінювання якості за рубрикою

Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	0	-	2
2. Формат	1	-	2
3. Стиснення	0	-	2
4. Код	1	-	2
5. Логіка	1	-	1
6. Українська	0	-	2
Разом /12	3	-	11

Розбіжність оцінок між двома прогонами(так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	так	-	ні
2. Формат	так	-	ні
3. Стиснення	так	-	ні
4. Код	так	-	ні
5. Логіка	так	-	так
6. Українська	так	-	ні

Відповіді на питання:

- 1. **Хто набрав найбільше?** Хмарна модель. Основна різниця виникла у логічних задачах (задача 5) та якості мови (задача 6).
- 2. **Розбіжність прогонів:** Розбіжності були у фактичних питаннях у локальних моделей. Якщо відповіді різні, модель галюцинує і не знає точної відповіді, а просто вгадує. Це ознака низької надійності.
- 3. **Вимога "без markdown":** Жодна модель не виконала вимогу ідеально – всі обгорнули JSON у маркдаун-блоки ````json ... ````.

4. **Проблеми з українською:** У локальних моделей зустрічалися кальки (наприклад, "стек відрізняється від черги тим, що...").

Приклад провалу:

Запит: Надай інформацію про Одеський національний університет імені І. І. Мечникова у форматі JSON: name, year, city. Без пояснень, без markdown, лише JSON.

Відповідь моделі:

```
{  
  "name": "Одеський національний університет імені І. І. Мечникова",  
  "year": 1865,  
  "city": "Одеса"  
}
```

Правильна відповідь: Мав бути просто сирий текст JSON без маркдаун обгортки.

Пояснення: Модель не витримала формат, оскільки у її навчальних даних всі JSON-фрагменти супроводжувалися маркдаун-обгорткою. Звичка виявилася сильнішою за пряму інструкцію.

Питання до чат-бота:

1. **Інші підходи до оцінювання:** LLM-as-a-Judge (одна велика модель оцінює іншу), автоматичні бенчмарки (MMLU, HumanEval) та людська розмітка (A/B тестування).
2. **Ігнорування markdown:** Це глибоко вкорінений патерн (bias) з етапу Instruction Tuning, який важко перебити звичайним промптом.
3. **Малі моделі та українська:** Українська мова становить мізерний відсоток у загальних датасетах, тому малим моделям не вистачає ємності (параметрів), щоб добре її засвоїти.
4. **Різна якість на однаковий запит:** Через температуру (temperature) – механізм випадкового вибору токенів.
5. **Галюцинації:** Це генерація правдоподібного, але хибного тексту. Модель не має "індикатора впевненості" і завжди намагається дати максимально ймовірне продовження тексту.

Власні думки: Оцінка за рубрикою досить надійна, хоч і вимагає часу. Найпоказовішим виявилось завдання на логіку (де qwen3:4b обійшла llama3.2:3b) та формат. Для роботи з українськомовним текстом я б однозначно обрав хмарну модель.

Завдання 6. Керування випадковістю (temperature і top_p)

Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	2	0
temperature	0.7	5	-
temperature	1.5	5	0
top_p	0.1	1	-
top_p	0.5	5	-
top_p	1.0	5	-

Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання:

- Температура для JSON:** 0.0 . Це гарантує детермінованість і строге дотримання структури без фантазій.
- Генератор заголовків:** 0.7 або вище. Це дасть креативність і різноманітність.
- Різниця top_p і temperature:** Temperature змінює самі ймовірності (робить розподіл більш плоским), а top_p просто відсікає маловрогідний "хвіст", зберігаючи відносні ймовірності лідерів незмінними.
- Чи гарантує 0.0 повну відтворюваність?** Мій експеримент показав 10/10 однакових, але загалом 100% гарантії немає через специфіку обчислень з рухомою комою на GPU.
- Найбезглуздіша відповідь:** "Рік заснування Одеська Політехніка - 2045". З'явилася, бо при $t=1.5$ модель з високою ймовірністю вибирає слова-аутсайдери (галюцинує), втрачаючи логічний зв'язок з реальністю.

Питання до чат-бота:

- Математика temperature:** Вона ділить логіти (сирі оцінки моделі перед softmax) на значення температури. Якщо $T < 1$, різниця загострюється, якщо $T > 1$ — згладжується.

- 2. **Механізм top_p:** Це nucleus sampling. Він сортує токени за спаданням ймовірності і відсікає ті, які виходять за межі накопиченої суми p.
- 3. **Одночасна зміна:** Зазвичай рекомендують змінювати щось одне (або temp, або top_p), щоб легше було контролювати та прогнозувати результат.
- 4. **Відсутність 100% гарантії:** Через неасоціативність паралельних матричних множень чисел із рухомою комою (floating point) на сучасних GPU.
- 5. **Практичні значення:** Для коду/даних (JSON): 0.0 - 0.2. Для звичайного чату чи написання листів: 0.7 - 0.8. Для брейнштормінгу: 1.0+.

Власні думки: Для отримання структурованих даних (JSON) в наступних лабораторних обов'язково треба ставити temperature=0, інакше агент колись обов'язково зламається через випадкову галюцинацію у синтаксисі. Налаштування "за замовчуванням" (зазвичай 0.7) абсолютно не підходить для програмної інтеграції.

Підсумкове порівняння моделей та Висновок

Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	Середня (~60-80 симв/с)	Низька	Дуже висока (~800+ симв/с)
TTFT	Найкраща (на теплому)	Добра	Залежить від інтернету (пінг)
Якість, балів /12	3	-	11
Стабільність формату (завдання 2)	Низька	Середня	Висока
Потрібні ресурси	~3 ГБ VRAM / RAM	~5 ГБ VRAM	Тільки інтернет
Вартість	Безкоштовно	Безкоштовно	Платна (або безкоштовні ліміти)
Куди йдуть ваші дані	Нікуди (локально)	Нікуди (локально)	На сервери провайдера
Працює без інтернету	Так	Так	Ні

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Коли обирати саме її	Для конфіденційних даних на слабкому ПК	Для конфіденційних даних на сильному ПК	Для високої якості та швидкості

Висновок:

Під час виконання лабораторної роботи я розгорнув локальне середовище з використанням Ollama та порівняв інструктивну (llama3.2:3b) і міркувальну (qwen3:4b) моделі. Також я підключив хмарну модель через OpenAI-сумісний API. Я дослідив вплив системних промптів на поведінку моделей, а також проаналізував залежність якості генерації від параметрів temperature і top_p .

Головне правило вибору: локальну модель слід обирати для роботи з приватною інформацією (комерційною таємницею) або коли немає доступу до інтернету, попри її меншу загальну ерудицію. Хмарну модель слід обирати в усіх інших випадках, оскільки вона забезпечує значно вищу надійність збереження формату (JSON), краще володіння українською мовою та вищу швидкість генерації довгих текстів.

Додатково варто відзначити, що робота з різними параметрами генерації (на кшталт temperature та top_p) наочно продемонструвала баланс між креативністю та точністю. Для суворих інженерних задач (наприклад, генерація формату JSON) критично важливо зводити випадковість до нуля (temperature=0), тоді як для творчих завдань підвищення температури дозволяє моделі розкрити свій потенціал. Загалом, практичний досвід розгортання локальних моделей підтвердив їхню життєздатність як інструменту для автономних та захищених систем, хоча для загальних завдань хмарні рішення наразі залишаються лідерами за швидкістю та якістю відповідей.